Smart MCQ Solver Challenge

Project Report -: Introduction to DL and GenAI Project

Roll No: 23f3000333 | Swati Singh

Abstract

This project builds an MCQ (multiple-choice question) answering system for Kaggle's Smart MCQ Solver Challenge, where each question has five options (A–E) and the goal is to rank the top three most likely correct answers, scored with MAP@3. Three models were built and compared: a fine-tuned DeBERTa-v3-base transformer, a deep MLP trained from scratch on TF-IDF features, and a third, bonus model using sentence-embedding similarity was also tried.

The most important finding of the project came from a bug, not a feature: the from-scratch MLP kept hitting near-100% validation accuracy, which turned out to be a sign of data leakage rather than good performance. Three real Kaggle submissions confirmed this every time the ensemble trusted that model more, the real leaderboard score got worse, even though the validation score looked better. The final submission uses DeBERTa alone, which was the only model with a believable, gradually-improving training curve (val MAP@3 ≈ 0.81), well above the 0.73 cutoff.

Introduction

2.1 Problem Statement

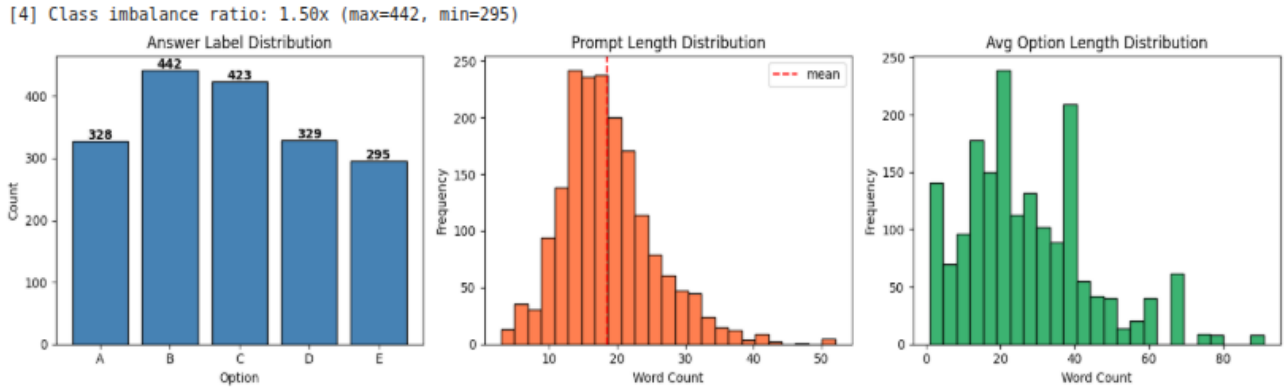
The competition gives a question prompt and five answer options per row and asks for a ranked top-3 prediction. It's scored with MAP@3: 1.0 if the correct answer is ranked first, 0.5 if second, 1/3 if third, 0 otherwise. The training set has 2,000 labeled rows; the test set has 500 unlabeled rows.

2.2 Objectives

Beat the 0.73 MAP@3 cutoff, build and fairly compare three different model types (pretrained, from-scratch, and one model of choice), track everything on WandB, and as it turned out correctly diagnose and respond to a data leakage problem rather than just chase a higher validation number.

Dataset & Preprocessing

After cleaning, the training data has 1,817 usable rows, split 90/10 into 1,635 training rows and 182 validation rows. The five answer positions are close to evenly distributed as correct answers, though B and C show up slightly more often than E, which is why the DeBERTa training uses a class-weighted loss.



Preprocessing steps

- Cleaned all text fields: collapsed extra whitespace, filled missing values with empty strings.
- Removed 183 exact-duplicate questions before splitting into train/validation this matters because if the same question ends up in both sets, the model can 'cheat' on validation by having memorized it during training, making the score look better than it really is.
- Used a stratified 90/10 split so both sets keep the same answer-class balance.
- No augmentation was used .

Tokenization Strategy

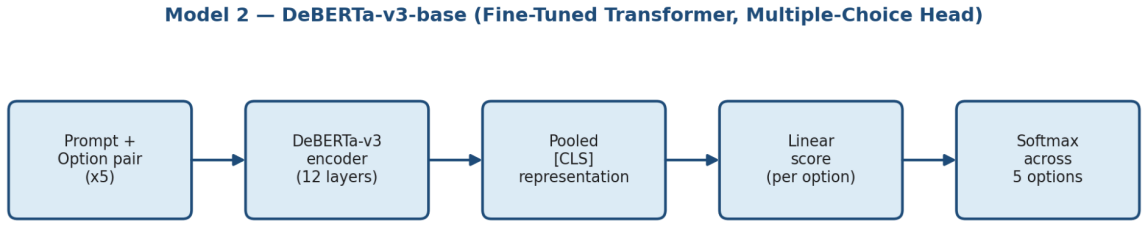
DeBERTa-v3 uses its own SentencePiece-based subword tokenizer, loaded via AutoTokenizer. Each question is tokenized as a paired input prompt as sentence one, each option as sentence two giving five token sequences per question, truncated/padded to 224 tokens, which is what HuggingFace's multiple-choice model format expects.

The two from-scratch models deliberately use simpler, independent tokenization so their errors aren't correlated with DeBERTa's: the MLP uses TF-IDF over 1-3 word n-grams (top 4,000 features), and the BiLSTM uses a small custom word tokenizer with a vocabulary built only from the training split (max 8,000 words), so no validation or test vocabulary leaks into it.

Modeling

5.1 DeBERTa-v3-base (Fine-tuned Transformer)

DeBERTa-v3-base is a 12-layer transformer that separates a token's content from its position ('disentangled attention') and was pretrained with an ELECTRA-style objective instead of plain masked-language-modeling both of which tend to make it a stronger starting point than a vanilla BERT for a given size. It's loaded through HuggingFace's AutoModelForMultipleChoice head, which scores each of the five options against the prompt and softmaxes across them.



Fine-tuned end-to-end (no frozen layers) for 5 epochs - 3 epochs clearly underfit at only 0.405 validation accuracy — with AdamW, a 1e-5 learning rate, cosine schedule, 10% warmup, effective batch size 16 (batch 4 × grad-accum 4), fp16, and gradient checkpointing to fit on a single Kaggle T4. A class-weighted cross-entropy loss corrects for the mild class imbalance in Section 3. No quantization was needed. Final validation performance: accuracy 0.6978, macro-F1 0.6950, MAP@3 0.8086

Figure 5.2a — Model 2 (DeBERTa) training loss, learning-rate schedule, and gradient norm across training steps, from WandB

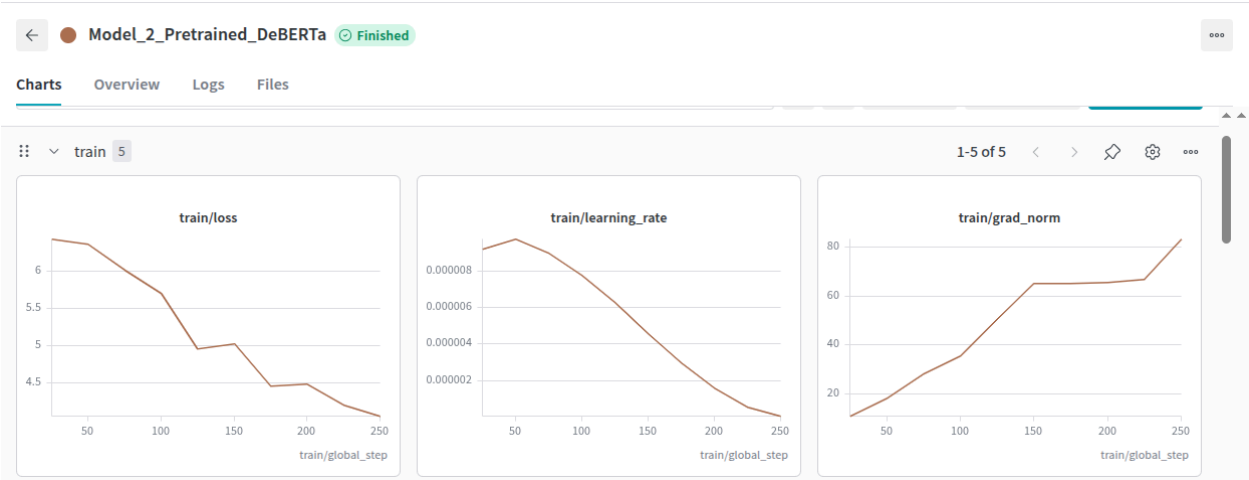


Figure 5.2b — Model 2 (DeBERTa) validation MAP@3, macro-F1, and loss across epochs — a gradual, believable improvement curve, unlike Model 1's flat line



5.2 Deep MLP (From Scratch)

A simple feed-forward network over TF-IDF features: Linear(4000→512) → LayerNorm → SiLU → Dropout(0.4) → Linear(512→128) → LayerNorm → SiLU → Dropout(0.2) → Linear(128→5). Trained 15 epochs with AdamW (lr=1e-3), keeping the checkpoint from whichever epoch had the best validation MAP@3, not just the last epoch.

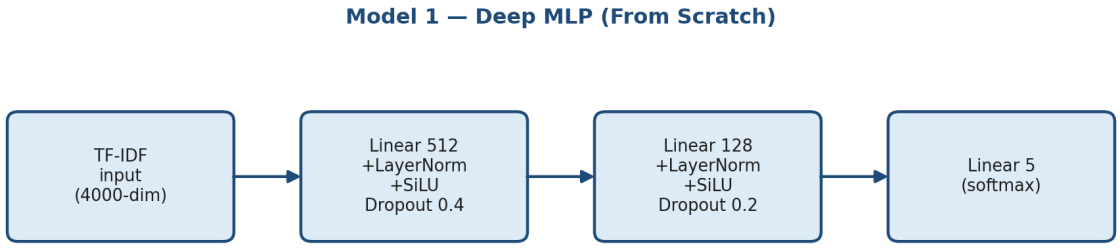


Figure 5.3 — Deep MLP architecture

This is the model behind the project's main finding. Even after removing duplicate questions, it hits ~100% training and validation accuracy in the first epoch — not realistic for a model learning MCQ reasoning from scratch on ~1,600 rows. This was confirmed with real evidence, not just suspicion: three actual Kaggle submissions gave this model increasing weight in the ensemble, and the real leaderboard score dropped every time (0.75644 → 0.749 → 0.7456 → 0.740), even as validation MAP@3 climbed toward 1.0. That gap between validation and the real leaderboard is the whole story.



Figure 5.4 — Model 1 (MLP) validation F1, validation accuracy, and training loss from WandB — note the flat line at 1.0 from the very first step, the signature of leakage rather than genuine learning

5.3 Sentence-Embedding Similarity + Logistic Regression

As an extra exploration beyond the three required models, prompt–option cosine similarity (from both SBERT embeddings and TF-IDF vectors) was fed into a logistic regression classifier. It trails every other model (MAP@3 0.489, accuracy 0.264), suggesting plain embedding similarity alone is a fairly weak signal for this task compared to a fully fine-tuned contextual model the options in this dataset apparently aren't distinguishable just by how similar their wording is to the prompt.



Figure 6.1 — Bonus similarity model's final MAP@3, macro-F1, and accuracy from WandB

Performance & Comparison

All four models are evaluated the same way — MAP@3 as the primary metric, plus accuracy and macro-F1 for a fuller picture — and all training runs are logged to the shared WandB project smart-mcq-solver-challenge.

Model Comparison — Smart MCQ Solver Challenge

Model	Accuracy	Macro-F1	MAP@3
DeBERTa-v3-base (fine-tuned)	0.6978	0.6950	0.8086
Deep MLP (from scratch)*	1.0000*	1.0000*	1.0000*
Similarity + LogReg	0.2637	0.2403	0.4890
Weighted Ensemble (analysis only)*	1.0000*	1.0000*	1.0000*

** Reflects the data-leakage issue diagnosed in the report (near-perfect score is not genuine skill).*

The MLP's near-perfect numbers reflect the leakage issue in Section 5.2, not genuine skill don't read this row as "the best model." DeBERTa's 0.81 val MAP@3 is the one to trust, since its curve improves gradually across epochs instead of jumping straight to perfect.

Kaggle Result

An unconstrained, validation-weighted ensemble across all four models was tried and rejected after three real submissions showed it reliably underperformed DeBERTa alone on the actual leaderboard. An early version of this ensemble weighting the scratch MLP at 0.2, DeBERTa at only 0.1, and the weak similarity model at 0.7 still produced a validation MAP@3, macro-F1, and accuracy of exactly 1.0, despite neither the MLP's individual overfitting nor the similarity model's weak 0.49 MAP@3 justifying that on their own. This is strong evidence that even a small weight on a leaking model can distort the blended ranking enough to look artificially perfect.

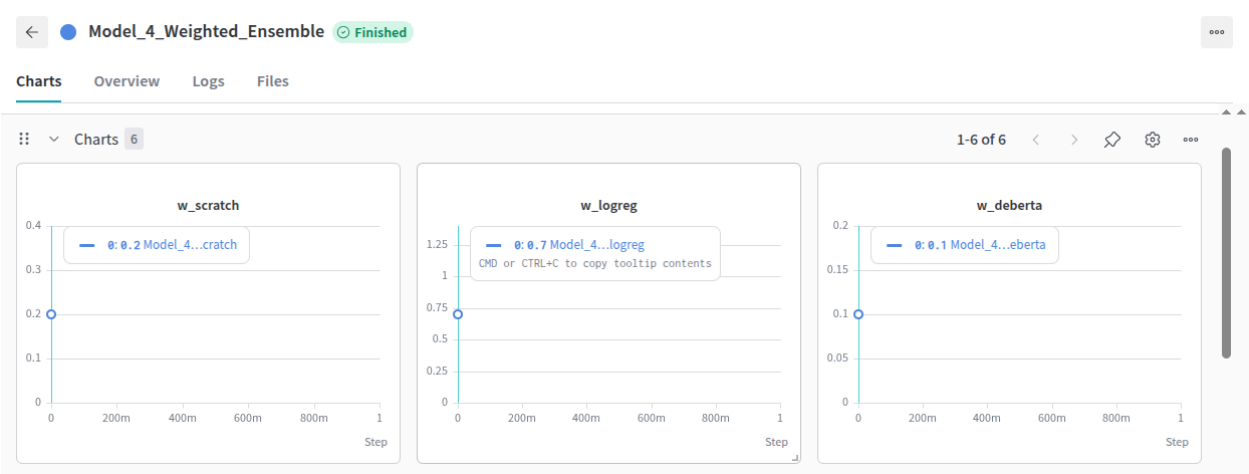


Figure 6.2 — An early ensemble's search weights (scratch, logreg, deberta) from WandB



Figure 6.3 — That same ensemble's validation MAP@3, macro-F1, and accuracy, all flat at 1.0

That weight-search is kept in the notebook as a labeled, analysis-only cell for the viva, but the final submission.csv is built from DeBERTa's predictions alone.

Conclusion & Future Work

Key Learnings

- A model scoring suspiciously well on validation deserves scrutiny before celebration, not after.
- A fine-tuned pretrained transformer beats both from-scratch models by a wide margin on a small (~1,800 row) dataset — the expected outcome, and a good demonstration of why transfer learning is the default choice for small-data NLP.

- The real leaderboard, not the offline validation split, is what actually proved the leakage hypothesis — three submissions getting worse while validation got better was the deciding evidence.

Challenges

The biggest challenge was the debugging process itself: three submissions in a row scored worse than the original 0.75644 baseline, even though each one looked better offline. Diagnosing it meant comparing the MLP's train-vs-validation gap, tracing how much ensemble weight it got across the three submissions, and eventually deciding to just exclude it rather than keep tuning around a broken signal.

Future Work

- Find the actual source of the MLP's leakage (near-duplicate but non-identical questions, or n-gram artifacts) instead of only working around it.
- Once Model 1 and the BiLSTM's generalization is better understood, revisit ensembling - a validated blend might still beat DeBERTa alone.
- Try a larger transformer (deberta-v3-large) with more compute.
- Add data augmentation (option shuffling, paraphrasing) to grow the training set.

8. References

Microsoft. DeBERTa-v3-base Model Card. Hugging Face Model Hub.

Sentence-Transformers. all-MiniLM-L6-v2 Model Card. Hugging Face Model Hub.

Weights & Biases. Experiment Tracking Documentation. wandb.ai.

Kaggle. Smart MCQ Solver Challenge — Competition Page.